

## ADR 0002 — Escolha de banco de dados

- **Status:** Aceito
- **Data:** 2026-06-23
- **Decisor:** dba (com `arq`, `back`)
- **Relacionado:** ADR 0001 — Plataforma de backend

### 1. Decisão

Usar **PostgreSQL** (gerenciado pelo Supabase, ver ADR 0001). A reserva sem duplicidade é garantida por **índice único parcial**; a privacidade por **RLS fail-closed**; metadados flexíveis (Open Graph de link de loja) em **JSONB**.

### 2. Por que relacional, e por que Postgres

O domínio do BdayList é um caso-livro de modelagem relacional: entidades de cardinalidade fixa ( `Evento→Lista→Presente`, `Convidado→Reserva→Presente` ) com **invariantes de concorrência** (reserva única) que exigem garantias transacionais fortes.

| Critério                                    | PostgreSQL                                 | MySQL (InnoDB)                 | MongoDB                                |
|---------------------------------------------|--------------------------------------------|--------------------------------|----------------------------------------|
| Modelo                                      | Relacional + JSONB híbrido                 | Relacional                     | Documento                              |
| ACID multi-tabela                           | Completo                                   | Completo                       | Multi-doc desde 4.0, com custo/limites |
| Concorrência                                | MVCC; <code>SERIALIZABLE</code> real (SSI) | MVCC; serializable via locking | Sem isolamento serializável entre docs |
| Índice único parcial ( <code>WHERE</code> ) | Sim                                        | Não                            | Existe, mas sem FK p/ o grafo          |
| Integridade referencial (FK/CHECK)          | Nativa                                     | FK nativo, CHECK só 8.0.16+    | <b>Sem FK</b> (vira código de app)     |
| RLS (acesso por linha)                      | Nativa                                     | Não tem                        | Não nativa                             |

**Por que documento custa caro aqui:** o agregado natural (Lista com presentes) é **escrito concorrentemente por muitos convidados** — exatamente onde embutir tudo num documento vira contenção e race no array. Você acabaria reconstruindo o relacional **sem FK**, **CHECK** e **RLS**.

**Por que não MySQL:** capaz de ACID, mas **não tem índice único parcial nem RLS** — as duas ferramentas que mais importam neste domínio.

### 3. Mecanismo de reserva sem duplicidade (anti-TOCTOU)

| Abordagem                                                | Garante?             | Concorrência                  | Veredito                         |
|----------------------------------------------------------|----------------------|-------------------------------|----------------------------------|
| Índice único parcial                                     | Sim, pelo banco      | Ótima (conflito só no commit) | <b>Padrão</b>                    |
| <code>SELECT ... FOR UPDATE</code> + <code>INSERT</code> | Sim, se sempre usado | Lock de linha                 | Complemento p/ presente em grupo |
| <code>SERIALIZABLE</code> sem constraint                 | Sim, mas com retries | Mais caro                     | Não como mecanismo primário      |

O índice único parcial vence porque a invariante fica **no schema**: qualquer caminho de escrita (app, RPC, admin) é forçado a respeitá-la — lock/isolamento dependem de o código lembrar.

```
-- presente comum: no máx. 1 reserva ATIVA; canceladas não bloqueiam re-reserva
create unique index uq_reserva_ativa_por_slot
  on reservas (presente_id, slot)
  where cancelada_em is null;

-- mesmo convidado não reserva 2x o mesmo presente (ativo)
create unique index uq_um_convidado_por_presente
  on reservas (presente_id, convidado_id)
  where cancelada_em is null;
```

A 2ª tentativa concorrente recebe **unique\_violation** no commit → idempotente, sem TOCTOU, sem lock manual. **Presente em grupo** usa cota/slot (a unicidade muda de "uma por presente" para "uma por slot"). Detalhe completo na [spec do motor de reserva](#).

#### 4. RLS resolve "surpresa" e PII nativamente

```
alter table reservas enable row level security;

-- host vê reservas do próprio evento SÓ se a lista não for surpresa
create policy host_ve_reservas on reservas for select
using (exists (
  select 1 from presentes p join listas l on l.id = p.lista_id
  join eventos e on e.id = l.evento_id
  where p.id = reservas.presente_id
    and e.host_id = auth.uid()
    and l.modo_surpresa = false
));
```

No app, cada query nova precisa lembrar de filtrar — uma rota esquecida vaza PII. Na RLS, a política é imposta pelo motor, **fail-closed por padrão**. Para uma SPA static export que fala direto com o Postgres, RLS é **pré-requisito**, não opção.

#### 5. Comparativo de managed Postgres (2026-06-23)

| Serviço                      | Free tier                                     | 1º pago             | Pooling                        | Backup / PITR                           | Scale-to-zero       |
|------------------------------|-----------------------------------------------|---------------------|--------------------------------|-----------------------------------------|---------------------|
| Supabase                     | 500 MB, 50k MAU, pausa 7d, sem backup no free | Pro US\$25          | PgBouncer incluso (porta 6543) | Diário no Pro; PITR add-on ~US\$100/mês | Não (pausa no free) |
| Neon                         | 0.5 GB, PITR 6h grátis                        | Launch US\$5 + uso  | pgBouncer                      | PITR 7d (Launch)                        | Sim                 |
| AWS Aurora Serverless v2     | sem free real                                 | ~US\$43–45/mês piso | RDS Proxy à parte              | PITR nativo                             | min 0 ACU           |
| Mongo Atlas * (contraponto)* | M0 512 MB                                     | Flex US\$8–30       | —                              | M0 sem backup                           | —                   |

**Leitura para MVP:** Supabase é o melhor \*fit de produto\* (RLS + Auth + pooler + Edge Functions num só lugar, casando com o static export). Neon é a rota de fuga mais barata se custo de PITR/egress pesar. Aurora só com SLA/escala que o MVP não tem.

#### 6. Riscos e mitigação

| Risco                              | Mitigação mais simples                                                                          |
|------------------------------------|-------------------------------------------------------------------------------------------------|
| Race / reserva dupla               | Índice único parcial (banco impõe; app trata <code>unique_violation</code> como "já reservado") |
| Slot preso após cancelar           | <code>cancelada_em</code> (soft-cancel) no <code>WHERE</code> do índice                         |
| Vazamento de PII / surpresa        | RLS fail-closed; nunca <code>service_role</code> no front                                       |
| Status do presente dessincronizado | Status <b>derivado</b> das reservas (view/trigger), não armazenado solto                        |
| Exaustão de conexões (SPA→DB)      | Usar <b>pooler/PgBouncer</b> (porta 6543), não conexão direta                                   |
| Token de share adivinhável         | <code>gen_random_bytes(16)</code> = 128 bits                                                    |

---

## Fontes (2026-06-23)

[Supabase pricing](#) · [Neon pricing](#) · [Aurora pricing](#) · [MongoDB pricing](#)